

C++ Programming: Problem Solving and Programming

Chapter 11 Program Design

The Need for Good Design

```
include <stdlib.h>
it digit_count(int n);
it Base=1000; // Base for calculation //
**** Main ****
it main ()

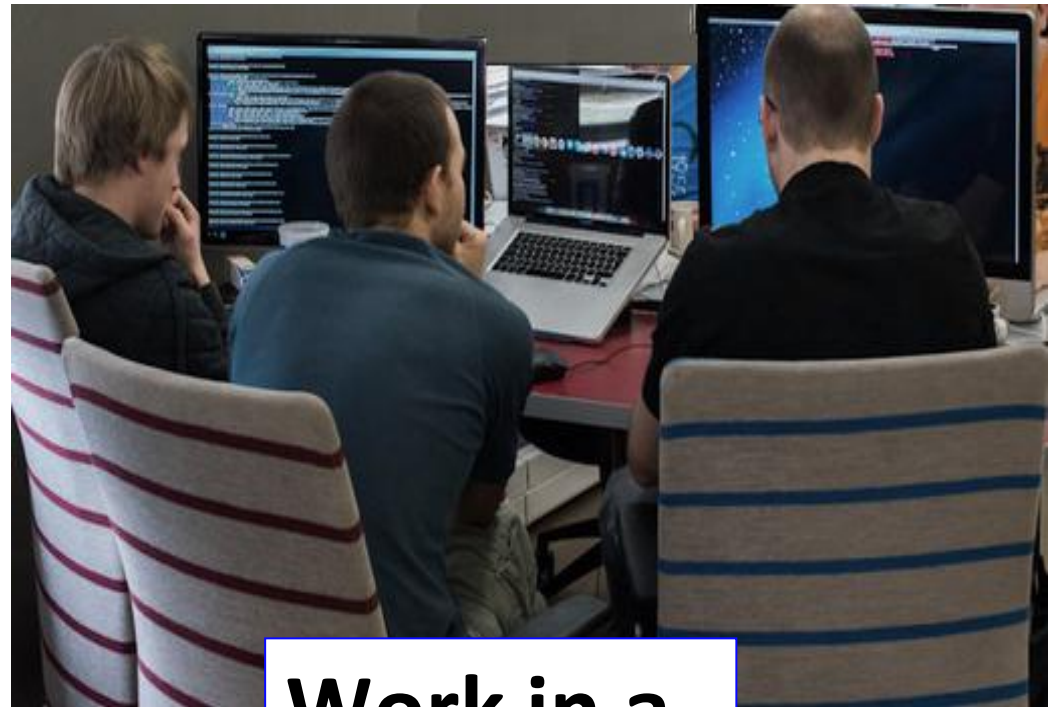
int i,j,k,n,N,l_arr=1,carry,count=0,length,size;
int *fibo=NULL, *arr1=NULL, *arr2=NULL, *temp;
printf("Program to find n-th Fibonacci Number.")
printf("\nEnter n: "); scanf("%d",&N);
if(N<0) n=-N; else n=N;

/* Initializing arrays */
temp = (int*) realloc (fibo, l_arr*sizeof(int));
fibo=temp; fibo[0]=0;
temp = (int*) realloc (arr1, l_arr*sizeof(int));
arr1=temp; arr1[0]=0;
temp = (int*) realloc (arr2, l_arr*sizeof(int));
arr2=temp; arr2[0]=1;

/* Loop for calculating Fibonacci numbers */
for(j=2;j<=n;j++){
    carry=0;
    /* Loop for adding each array element */
    for(i=0;i<l_arr;i++){
        fibo[i]=arr1[i]+arr2[i]+carry;
        if(fibo[i]>9){
            carry=fibo[i]/10;
            fibo[i]=fibo[i]%10;
        }
        temp = (int*) realloc (arr1, l_arr*sizeof(int));
        arr1=temp; arr1[l_arr-1]=fibo[i];
        temp = (int*) realloc (arr2, l_arr*sizeof(int));
        arr2=temp; arr2[l_arr-1]=fibo[i];
        l_arr++;
    }
    /* Updating array elements */
    for(k=0;k<l_arr;k++){
        arr1[k]=arr2[k]; arr2[k]=fibo[k];
    }
    count++;
}

sizeofint count(Base=1; // Size of elements //
```

**Large and
Complicated
Program**



**Work in a
team**

Stores Components Separately

- Stores long and complicated program in separate files

Reusability

- Reuse in other program

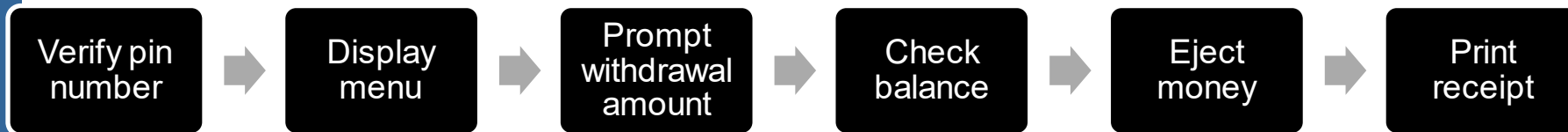
Maintainability

- Easier to manage

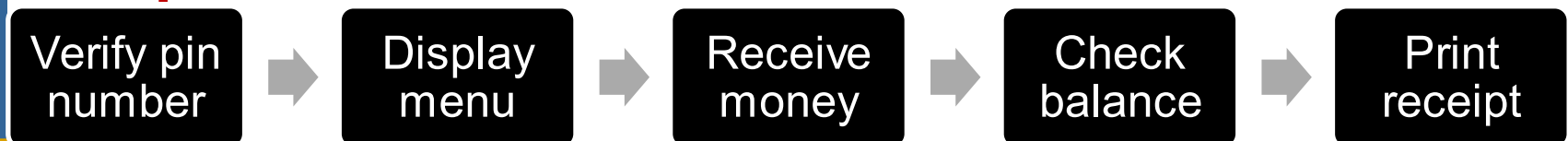
Example

What are repeated in the following transaction?

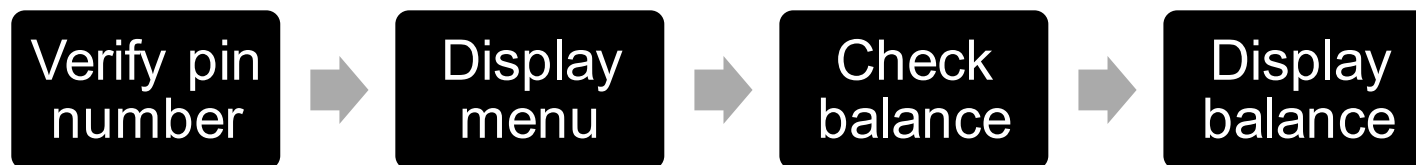
Withdrawal:



Deposit:



Balance Inquiry:



Inclusion of components

- To include components required (like functions) from separate files
- General terms used:
 1. Include
 2. Import
 3. copy

Variable and function names

General guidelines :	Good	Bad
Use meaningful names		s
Use pronounceable names		xfqb
Use suitable abbreviations		st
Avoid digits in names		menu1
Separate words clearly		dotwice
Use verbs for flags (variables holding a status)		approval

Orgnization of Modules

- Too many or too few instructions in a module
 - Makes the program hard to follow
 - Reduces flexibility
- Two rules to follow are:
 - Reduce **coupling**
 - Increase **cohesion**

Coupling

- The measure of strength for the connection (relation) between 2 modules

- **Tight** coupling
 - Modules are closely related to one another
- **Loose** coupling
 - Modules are generally unrelated

Coupling

- When is the coupling tight?
 - If a module changes a variable value and other modules are affected (e.g. when using global variable)
 - If two modules can access the same variables

Tight Coupling → Not Good

Tight Coupling

- Much dependence between modules
- Makes programs more error prone
 - Many data paths to keep track of
 - Many chances for bad data to pass from one module to another
 - Many chances for one module to alter data needed by another module

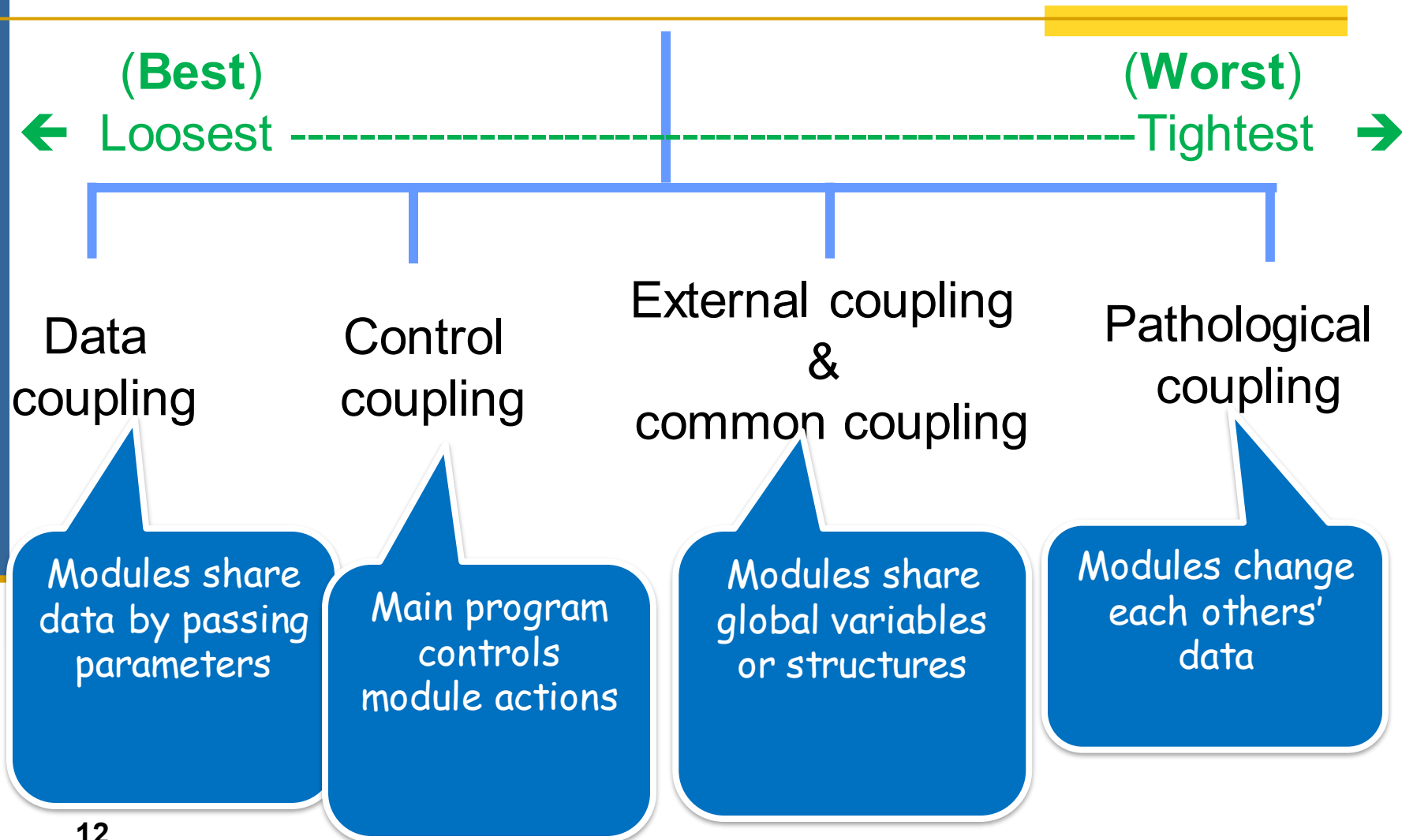
Coupling

- When is the coupling loose?
 - If a module changes do not affect other modules
 - If data is shared by passing variables to other modules

Loose Coupling → Good

- Should reduce coupling
 - easier to write, maintain and reuse

Level of Coupling



Level of Coupling



(Best)

← Loosest ---

Data
coupling

Modules share
data by passing
parameters

```
processCustomerRecord
```

```
:      :
```

```
final = calculateFinalPrice( totalPrice , salesTax )
```

```
:      :      :
```

```
:
```

```
END
```

```
calculateFinalPrice(double price, double tax)
```

```
:
```

```
finalPrice = price + (tax * price)
```

```
return finalPrice
```

```
END
```

Level of Coupling



(Best)



Loosest

Control
coupling

Main program
controls
module actions

```
processInputCode
  Read inputCode
  chooseAction (inputCode)
  :
END
```

```
chooseAction (inputCode)
  CASE OF inputCode
    1: Read employee record
    2: Print page heading
    3: Open employee master file
    4: Set pageCount to zero
    5: errorMsg = "Invalid code"
  ENDCASE
```

Level of Coupling



salesTax=0

calculateSalesTax

IF productPrice < \$10.00 THEN

salesTax = productPrice * 0.05

ELSE

IF productPrice < \$50.00 THEN

salesTax = productPrice * 0.25
:

ENDIF

END

global variable

calculateAmountDue

:

amountDue = totalAmount + **salesTax**

END

(Worst)

Tightest →

al &
coupling

Modules share
global
variables or
structures

```
typedef struct{
    char name[31], id[11];
    double salary;
} Employee;
Employee emp;

void readEmpRecord()
{
    ...
    while (!feof(fptr)) {
        outfile << emp.ID << emp.name << emp.salary;
        ...
    }
}

int validateEmpSalary() {
    if (emp.salary < 0)
        cout << "Invalid salary!\n";
    ...
}
```

global structure variable



Level of Coupling



calculateSalesTax

IF product is sales exempt THEN
 salesTax = 0

amountDue = 0

ELSE

 IF productPrice < \$50.00 THEN

salesTax = productPrice * 0.25

 :

 ENDIF

END

amountDue
doesn't "belong"
to this module,
but it is being
modified here

salesTax doesn't
"belong" to this
module, ...

calculateAmountDue

:

amountDue = totalAmount + **salesTax**

salesTax = 0

END

(Worst)

-----Tightest →

Pathological
coupling

Modules change
each others'
data

Orgnization of Modules

- Two rules to follow:

- Reduce **coupling** ↓

- Increase **cohesion** ↑

Cohesion

- The measure of internal strength (independence) of a module

- **Strong/high** cohesion

- All statements serve to achieve the purpose of the function

- **Weak/low** cohesion

- All statements are not related to the purpose of the function

Weak Cohesion



Cohesion

- When is the cohesion strong?
 - If all operations are related to the function
 - If modules can access the same variables

Strong Cohesion → Good ...

- Cohesion programs are reliable
 - Stronger, less relations with other modules
 - Easier to write, read, and maintain

Levels of Cohesion

← strongest (Best) ----- weakest (Worst) →

Functional cohesion

Sequential cohesion

Communicational Cohesion

Temporal cohesion

Procedural cohesion

Logical cohesion

Coincidental cohesion

Perform only 1 task

Performs fewer tasks;

tasks are quite related

Tasks are often in sequence

data are shared

Performs more tasks

Tasks are less or not related;

Tasks may not be in sequence

Data are not shared

Perform many tasks; totally not related

Level of Cohesion

1. Functional cohesion

calculateSalesTax

IF product is sales tax exempt

salesTax = 0

ELSE IF product_price < \$ 50.00

salesTax = productPrice * 0.25

ELSE IF product_price < \$100.00 THEN

salesTax = productPrice * 0.35

ELSE

salesTax = productPrice * 0.5

ENDIF

Performs only 1
task

Level of Cohesion

2. Sequential cohesion

```
processPurchases
  totalPurchase = 0
  Get numberOfPurchases
  DO loopIndex = 1 to numberOfPurchases
    Get purchase
    Add purchase to totalPurchase
  ENDDO
  salesTax = totalPurchase * salesTaxPercent
  amountDue = totalPurchase + salesTax
```

performs 3 tasks in sequence
on the same data

Level of Cohesion

3. Communicational cohesion

```
IF productType NOT = '0' THEN  
    errorFlag = TRUE  
    errorMessage = 'invalid product type'  
    printErrorReport  
ENDIF
```

```
IF productNo is NOT numeric THEN  
    errorFlag = TRUE  
    errorMessage = 'invalid product no'  
ENDIF
```

```
IF productNo = blanks OR productNo has leading blanks  
    errorFlag = true  
    errorMessage = 'invalid product no'  
    printErrorReport  
ENDIF
```

Performs 3 related tasks not necessarily in sequence but on the same data

Level of Cohesion

4. Temporal cohesion

Initialization

Open transaction file

Issue prompt 'Enter todays date ---- DDMMYY'

Read todaysDate

Set transaction record

Read transaction record

IF not EOF

Increment transactionCount

ENDIF

Open report file

printPageHeading

Set reportTotal to zero

END

- performs more than one tasks
- these are related by the fact that they must occur within the **same time span**.

Level of Cohesion

4. Temporal cohesion

```
void initialization() {  
    sales = 0;  
    tax = 0;  
    total = 0;  
    printf("Monthly sales report:");  
    fptr1=fopen("sales.txt", "r");  
    fptr2=fopen("transaction.txt", "w");  
    printf("Enter date: ");  
    scanf("%d-%d-%d", &day, &mth, &year);  
    printf("Enter name: ");  
    scanf("%[^\\n]", name);  
    ...  
}
```

- performs more than one task
- these are related by the fact that they must occur within the **same time span**.

Level of Cohesion

5. Procedural cohesion

readStudentRecords_And_TotalStudentAges

Set numberOfRecords to zero

Set totalAge to zero

Read student record

DOWHILE more records exist

 Add age to totalAge

 Add 1 to numberOfRecords

 Read student record

ENDDO

Print numberOfRecords, totalAge

END

- Performs mainly 2 unrelated tasks
- In particular sequence
- On different data

Level of Cohesion

6. Logical cohesion

```
readAllFiles (fileCode)
```

```
  CASE of fileCode
```

```
    1:  Read customer transaction  
        IF not EOF, Increment customerTransactionCount  
        ENDIF
```

```
    2.  Read customer master record  
        IF not EOF, Increment customerMasterCount  
        ENDIF
```

```
    3.  Read product master record  
        IF not EOF, Increment productMasterCount  
        ENDIF
```

```
  ENDCASE
```

```
END
```

- Performs 3 possible unrelated task(s)
- Depends on a decision on different data

Level of Cohesion

7. Coincidental cohesion

fileProcessing

Open employee updates file

Read employee record

printPageHeading

Open employee master file

Set pageCount to one

Set errorFlag to false

Print error message

END

Instructions here
have no meaningful
relationship with
each other

Example

Characteristics			Type of cohesion
The module performs a single task.			(i) Function cohesion
The module does not perform a single task.	Data relates the activities inside the module.	Sequence is important.	(ii) Sequential cohesion
		Sequence is not important.	(iii) Communicational cohesion
	Logic control relates the activities inside the module.	Sequence is important.	(iv) Procedural cohesion
		Sequence is not important.	(v) Logical cohesion
	Neither data nor logic relates the activities inside the module.	Activities are related by time.	(vi) Temporal cohesion
		Activities are not related by time.	(vii) Coincidental cohesion

Organizing Modules: Cohesion



How the statements of a module serve to accomplish its purpose

Programming
rules:



INCREASE
Cohesion



Levels of Cohesion

⑦



BEST



WORST

Organization of Modules: Coupling



Coupling is the measure of the **strength** of the connection of two modules

Programming rules:

REDUCE
Coupling



Category / Type

Tight
Coupling



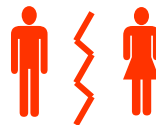
Depend
on other function



Loose
Coupling



Not depend
on other function



Levels of Coupling

④

Data coupling

Control coupling

External coupling
and
common coupling

Pathological
coupling

Good programming habits

- Plan before coding
- Use flowcharts or pseudocode to design your programs for ease of coding
- Desk check your program
 - It is the process of manually checking the logic of the program before coding them
 - This usually happens after we have finished the pseudo-code but before program coding
- Design programs with high cohesion and loose coupling for ease of maintenance and understanding